AUTOSAR COM Stack

Agenda

- Introduction
- CAN
- CAN Driver Module
- CAN IF Module
- CanTP Module
- PduR Module
- COM Module

PDUR Module

- The PDU Router module provides services for routing of I-PDUs (Interaction Layer Protocol Data Units) using the following module types:

  - Communication interface modules e.g. Com, IPduM, LinIf, CanIf, CanNm, FrIf and FrNm
  - Transport Protocol modules e.g. J1939Tp, LinTp (part of LinIf), CanTp, FrTp, Com, Dcm.

- The most common combination of upper and lower layer pairs is listed below:

  - Diagnostic Communication Manager (Dcm) and Transport Protocol modules,
  - Com and communication interface modules, Transport protocol modules or I-PDU Multiplexer,
  - I-PDU Multiplexer and communication interface modules.

- Above PDUR, all PDUs are protocol independent, and below PDUR all PDUs are routed to the protocol specific modules.

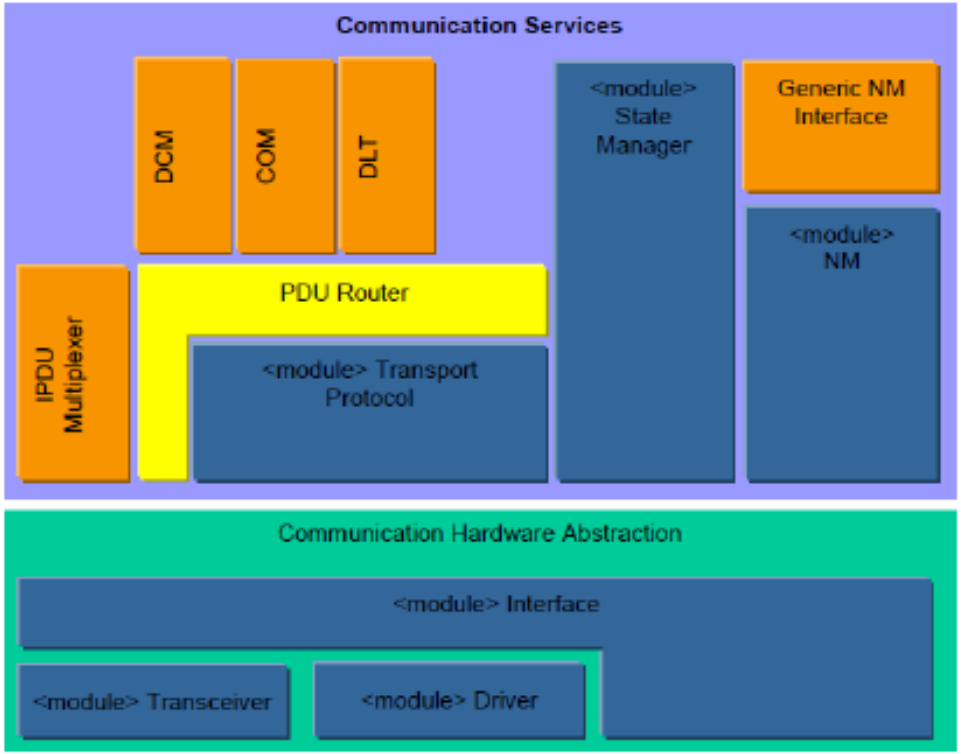


Figure 1.1: CommunicationStructure

PDU Router module function overview

The PDU Router module mainly consists of two parts:

- **PDU Router routing paths:**
  - Static routing paths describing the routing attributes for each I-PDU to be routed.

- **PDU Router Engine:**

  - Route the I-PDU from source to destination(s)
  - Translating the source I-PDU ID to the destination I-PDU ID
    (e.g. PduR_Com Transmit to CanIf_Transmit, PduR_CanIfTxConfirmation to Com_TxConfirmation).

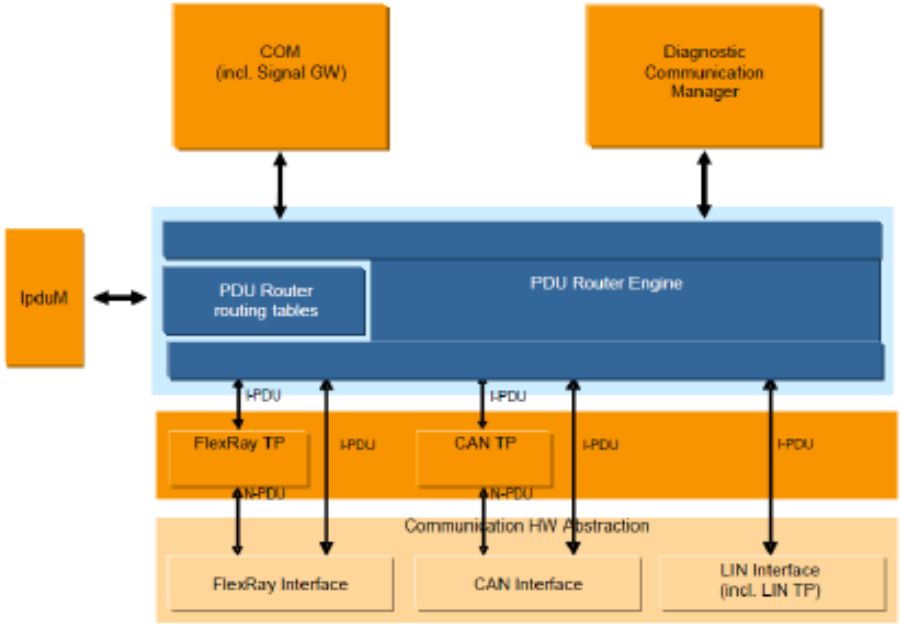


Figure 1.2: Detailed PDU Router Structure showing FlexRAy, CAN and LIN

PDU Router module operations

- **PDU Reception to local module(s): receive I-PDUs from lower layer modules** and forward them to one or more upper layer module

- **PDU Transmission from local module(s): transmit I-PDUs to one or more lower** layer modules on request of upper layer module

- **PDU Gateway:**

  -> Receive I-PDUs from an interface module and transmit the I-PDUs immediately via the same or other communication interface module(s)

  -> Receive I-PDUs from a transport protocol module and transmit the I-PDU via the same or other transport protocol module(s).

The PDU Router module can:

- **Singlecast (1:1**) an I-PDU from a local module to a communication interface module.
- **Multicast (1:n**) an I-PDU from a local module to communication interface modules.
- **Singlecast (1:1)** an I-PDU (Single Frame (SF) or Multiple frames (FF and CFs)) from a local module to a transport protocol module.
- **Multicast (1:n)** an I-PDU (Single Frame (SF)) from a local module to transport protocol modules.
- **Singlecast (1:1)** an I-PDU from a communication interface module to a local module.
- **Multicast (1:n)** an I-PDU from a communication interface module to local modules.
- **Singlecast (1:1)** an I-PDU (Single Frame (SF)) from a transport protocol module to local module.
- **Multicast (1:n)** an I-PDU (Single Frame (SF)) from a transport protocol module to local modules.
- **Gateway (1:1)** an I-PDU from a communication interface module to a communication interface module using last-is-best buffer.
- **Gateway (1:n)** an I-PDU from a communication interface module to communication interface modules using last-is-best buffer.
- **Gateway (1:1)** an I-PDU from a communication interface module to a communication interface module using FIFO.
- **Gateway (1:n)** an I-PDU from a communication interface module to communication interface modules using FIFO.
- **Gateway (n:1)** an I-PDU from communication interface modules to a communication interface module using FIFO.
- **Gateway (1:1)** an I-PDU from a transport protocol module to a transport protocol module using FIFO.
- **Gateway (1:n)** an I-PDU from a transport protocol module to transport protocol modules using FIFO.
- **Gateway (n:1**) an I-PDU from transport protocol modules to a transport protocol module using FIFO.

I-PDU handling

I-PDUs are identified by static I-PDU IDs.

- The PDU Router module determines the destination of an I-PDU by using the I-PDU ID in a static configuration table.
- I-PDUs are used for the data exchange of the modules directly above the PDU Router module, e.g. the Com module and Dcm module.
- The I-PDU ID is set in the configuration that also implements the API
- This will allow an efficient implementation of look-up tables in each module receiving an I-PDU ID (e.g. the PDU Router module’s configuration contains the I-PDU ID for the PduR_CanIfTxConfirmation).

I-PDU handling

- The PDU Router module shall transfer an I-PDU without modification in a consistent manner from the source module to the destination module(s).

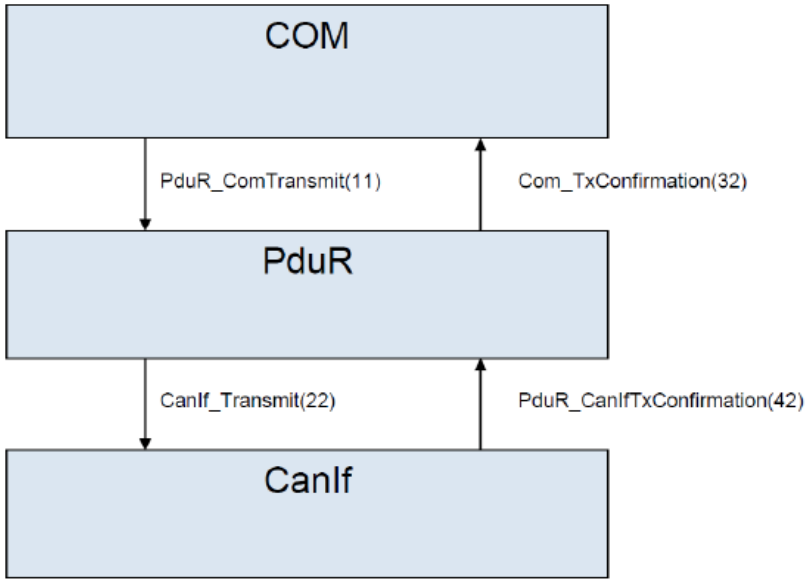


Figure 7.1: I-PDU ID Example

I-PDU Reception to upper module(s)

- The receive operation of the PDU Router module is always be finalized by an indication (PduR_<User:Lo>RxIndication or PduR_<User:LoTp>RxIndication) from a lower layer module (communication interface module or transport protocol module).
- The indication function is executed by the lower layer either in the context of a cyclic function after polling a communication driver or in the context of an interrupt.

Communication Interface

- The source communication interface module indicates a received I-PDU by calling PduR_<User:Lo>RxIndication. The I-PDU may have multiple destination local modules as configured by the routing path.

- The PDU Router module shall provide 1:n routing for an I-PDU received from a communication interface module and routed to one or more upper layer module(s).
  Example: An I-PDU is received on CanIf and forwarded to Com.

Transport Protocol

- In case of the transport protocol module the PDU Router module is first notified with a start of reception notification when receiving a first frame (FF) or single frame (SF). This call is be forwarded to the related upper layer module by calling **<Up>_StartOfReception.**

- The payload of each segment (N-PDU) is to be copied in the destination upper layer module by the within subsequent **<Lo>_CopyRxData** calls.

- After reception of the last N-PDU the transport protocol module will indicate the PDU Router module that the complete I-PDU has been received and the PDU Router module will forward this indication to the related upper layer module by calling **<Up>_TpRxIndication.**

Handling I-PDUs with unknown length

- The PduR is able to handling unknown length I-PDUs (i.e. streaming type of data) using the TP API. The definition of unknown length is indicated by TpSduLength=0.

- In a local receive situation and PduR_<User:LoTp>StartOfReception is called with TpSduLength=0, PduR shall call <Up>_StartOf Reception with TpSduLength=0.

I-PDU Transmission from upper module(s)

- The transmit operations of the destination lower layer modules are always asynchronous. This means that a transmission service request returns immediately after the I-PDU has been passed by the PDU Router module to the destination lower layer module.

- The transmit operation of the PDU Router module is triggered by a PDU transmit request from a source upper layer source module and forwards the request to a destination lower layer module.

- If the PDU Router module is notified by destination lower layer modules via **PduR<User:Lo>TxConfirmation** (communication interface) or **PduR<User:LoTp>TxConfirmation** (transport protocol) after successful or failed transmission of the I-PDU.

Communication Interface

There are three ways that I-PDUs can be transmitted on the communication interface:

1. **Direct data provision** - where the upper module is calling the PduR_<User:Up>Transmit function, the PDU Router module forwards the call to **<Lo>Transmit** and the data is copied by the lower communication interface module in the call.

2. **Trigger transmit provision** - where the lower communication interface module requests transmission of an I-PDU by using the **PduR<User:Lo>TriggerTransmit**, and PDU Router module forwards the call to **<Up>TriggerTransmit** and the data is copied by the upper module.

3. Where the upper module is calling the **PduR<User:Up>Transmit** function, the PDU Router module forwards the call to **<Lo>_Transmit** and the data is not copied by the lower module (communication interface module). The data will later be requested by the lower layer using **PduR<User:Lo>TriggerTransmit.**

Trigger transmit data provision

- The PDU Router module shall forward a **PduR<User:Lo>TriggerTransmit** request by the communication interface module to the upper module by calling **<Up>TriggerTransmit**.

- The PDU Router module shall copy the return value from **the <Up>TriggerTransmit** to the lower layer module.

Transport Protocol

- Initiation of transmission of I-PDU is made by a **PduR_<User:Up>Transmit** request by an upper layer source module. The PduR will forward the request to one or more destination lower layer transport protocol modules using <Lo>_Transmit according to the routing paths. Note that the <Lo>_Transmit may or may not contain data.

- The destination module(s) will request data by calling the **PduR_<User:LoTp>CopyTxData**.

- Retransmission (if supported by the transport protocol) of data is made by the **RetryInfoType** parameter.

- Finalizing the transmission the destination module(s) calls the **PduR_<User:LoTp>TxConfirmation**, which is forwarded to the source upper layer module.

I-PDU Gateway

- I-PDUs may be gatewayed from a source communication interface module to one (1:1) or more destination communication interface modules (1:n I-PDU gateway)
  – **For each destination the PDU Router module may buffer each destination of** an I-PDU in configurable depth (i.e. FIFO if more than one I-PDU).
  – An I-PDU may be received by an upper layer module in the same time as gatewayed to n destination communication interface(s).

- I-PDUs transported using TP may be gatewayed to one or more destination TP modules, with following scope:
  – **Both Single Frames and Multi Frames may be gatewayed to more than one** destination TP module or local module (e.g. Dcm).
  – **I-PDU transported in multiple N-PDUs may be routed to one destination in a** "on-the-fly gatewaying" manner, meaning that complete I-PDU does not need to be received before starting transmission on the destination TPmodule
  – **I-PDU transported in multiple N-PDUs may be gatewayed to another TP** module or received by a local module, not both.
  – **I-PDUs transported using TP module may be FIFO buffered. This applies to** both Single Frame and Multi Frame I-PDUs.

- I-PDUs can only be gatewayed between communication interface modules or TP modules, not a mix of them. For example an I-PDU cannot be received from CanIf and gatewayed to LinTp.

Cancel transmission

- An upper layer module may request cancellation of an I-PDU. The PDU Router module will forward the request to either one destination module (singlecast) or multiple destination modules (multicast).

- The PduR_<Up>CancelTransmit is used to cancel communication interface I-PDU and to cancel transport protocol I-PDUs in the case of forwarding.

Cancel reception

- An upper layer module may request cancellation of an I-PDU transported on transport protocol module(s). The PDU Router module will get a request through the PduR_<Up>CancelReceive. The confirmation of the cancellation request is made through the return value of <LoTp>_CancelReceive that is forwarded to the upper layer module as return value of PduR_<Up>CancelReceive.

State Management

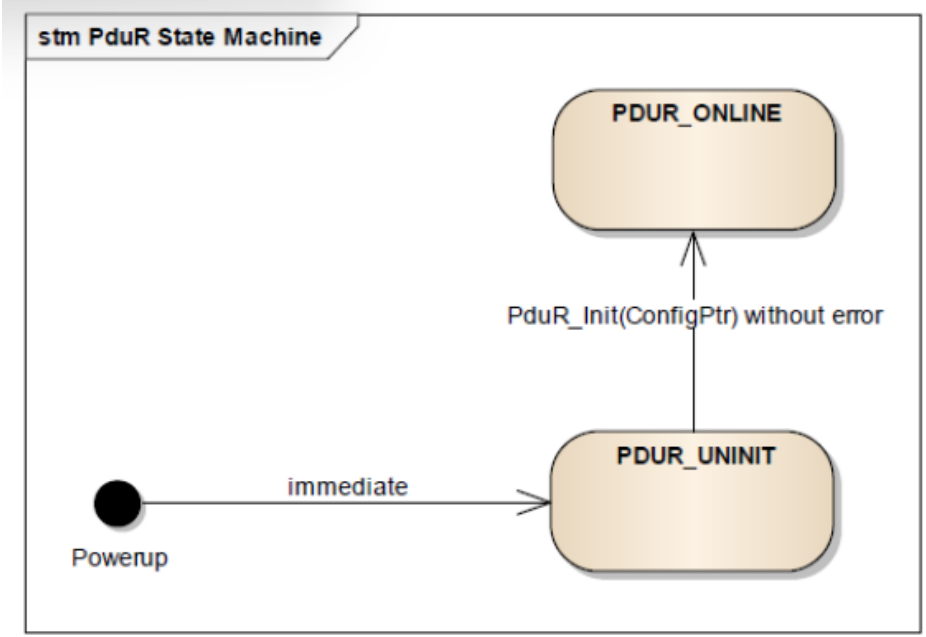


Figure 7.2: PDU Router states

Routing path groups

- A routing path group is a group of routing paths that can be disabled and enabled during runtime.
- This allows for enabling/disabling a set of routing paths related to a certain network or a certain kind of PDUs.
- Enabling and disabling of routing path groups is typically used by the BswM module.
- Routing path group can be disabled by the call to api PduR_DisableRouting()
- Enabling of I-PDU routing path groups shall be immediate

Error classification

Development Errors

Type of error	*Related error code*	*Error value*
Invalid configuration pointer	PDUR_E_INIT_FAILED	0x00
API service (except PduR_GetVersionInfo) used without module initialization or PduR_Init called in any state other than PDUR_UNINIT	PDUR_E_UNINIT	0x01
Invalid PDU identifier	PDUR_E_PDU_ID_INVALID	0x02
If the routing table is invalid that is given to the PduR_EnableRouting or PduR_DisableRouting functions	PDUR_E_ROUTING_PATH_GROUP_ID_INVALID	0x08
Null pointer has been passed as an argument	PDUR_E_PARAM_POINTER	0x09

Runtime Errors

Type of error	*Related error code*	*Error value*
TP module rejects a transmit request for a valid PDU identifier	PDUR_E_TP_TX_REQ_REJECTED	0x03
Loss of a PDU instance (buffer overrun in gateway operation)	PDUR_E_PDU_INSTANCES_LOST	0x0a

Function definitions

PduR_Init	void PduR_Init **(** const PduR_PBConfigType***** ConfigPtr)
PduR_GetVersionInfo	void PduR_GetVersionInfo **(** Std_VersionInfoType***** versionInfo)
PduR_GetConfigurationId	PduR_PBConfigIdType PduR_GetConfigurationId **(** void**)**
PduR_EnableRouting	void PduR_EnableRouting **(** PduR_RoutingPathGroupIdType id**)**
PduR_DisableRouting	void PduR_DisableRouting **(** PduR_RoutingPathGroupIdType id, boolean initialize**)**
PduR_<User:Up>Transmit	Std_ReturnType PduR_<User**:**Up>Transmit **(** PduIdType TxPduId, const PduInfoType***** PduInfoPtr **)**
PduR_<User:Up>CancelTransmit	Std_ReturnType PduR_<User:Up>CancelTransmit **(**PduIdType TxPduId **)**
PduR_<User:Up>CancelReceive	Std_ReturnType PduR_<User:Up>CancelReceive **(** PduIdType RxPduId **)**
PduR_<User:Lo>RxIndication	void PduR_<User:Lo>RxIndication **(** PduIdType RxPduId, const PduInfoType***** PduInfoPtr**)**
PduR_<User:Lo>TxConfirmation	void PduR_<User:Lo>TxConfirmation **(** PduIdType TxPduId,Std_ReturnType result**)**

PduR_<User:Lo>TriggerTransmit	Std_ReturnType PduR_<User:Lo>TriggerTransmit (PduIdType TxPduId,PduInfoType* PduInfoPtr)
PduR_<User:LoTp>CopyRxData	BufReq_ReturnType PduR_<User:LoTp>CopyRxData (PduIdType id, const PduInfoType* info, PduLengthType* bufferSizePtr)
PduR_<User:LoTp>RxIndication	void PduR_<User:LoTp>RxIndication (PduIdType id,Std_ReturnType result)
PduR_<User:LoTp>StartOfReception	BufReq_ReturnType PduR_<User:LoTp>StartOfReception (PduIdType id,const PduInfoType* info, PduLengthType TpSduLength, PduLengthType* bufferSizePtr)
PduR_<User:LoTp>CopyTxData	BufReq_ReturnType PduR_<User:LoTp>CopyTxData (PduIdType id,const PduInfoType* info, const RetryInfoType* retry,PduLengthType* availableDataPtr)
PduR_<User:LoTp>TxConfirmation	void PduR_<User:LoTp>TxConfirmation (PduIdType id, Std_ReturnType result)

Scheduled functions

- As any PDU Router operation is triggered by an adjacent communication module the PDU Router does not require scheduled functions.

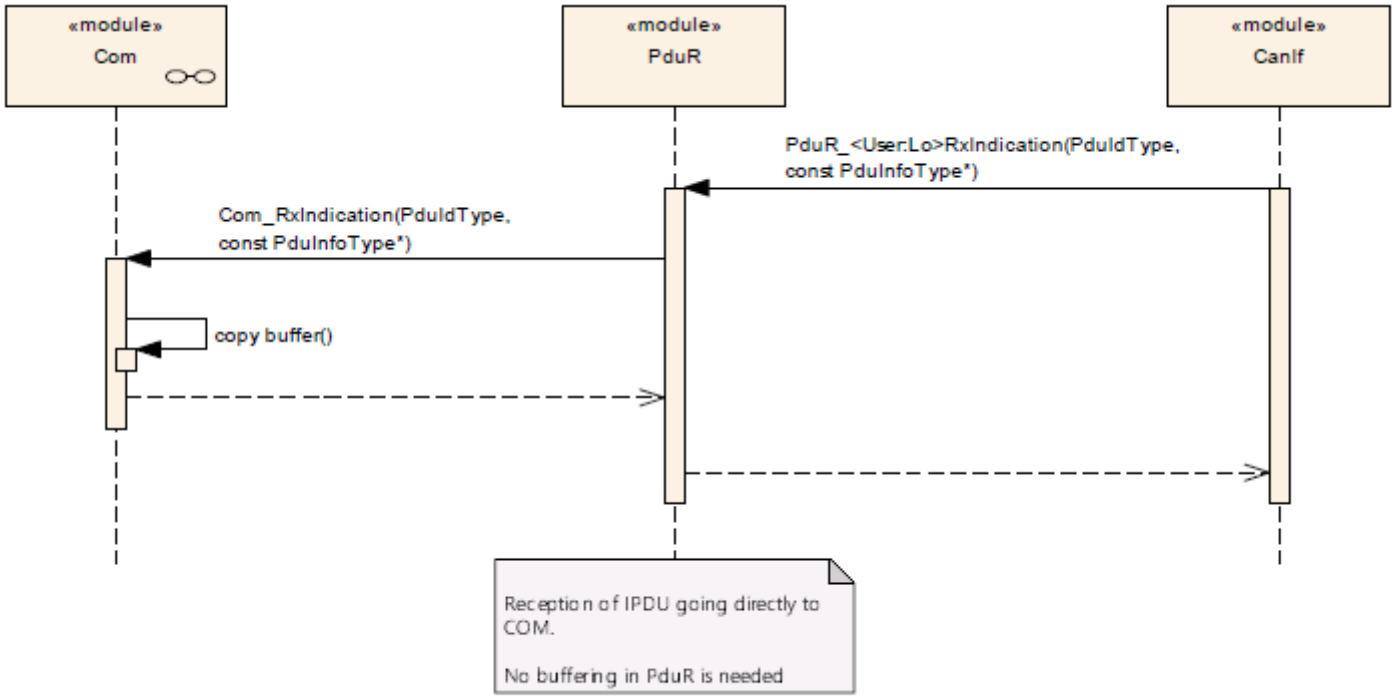


Figure 9.1: CanIf to COM I-PDU reception

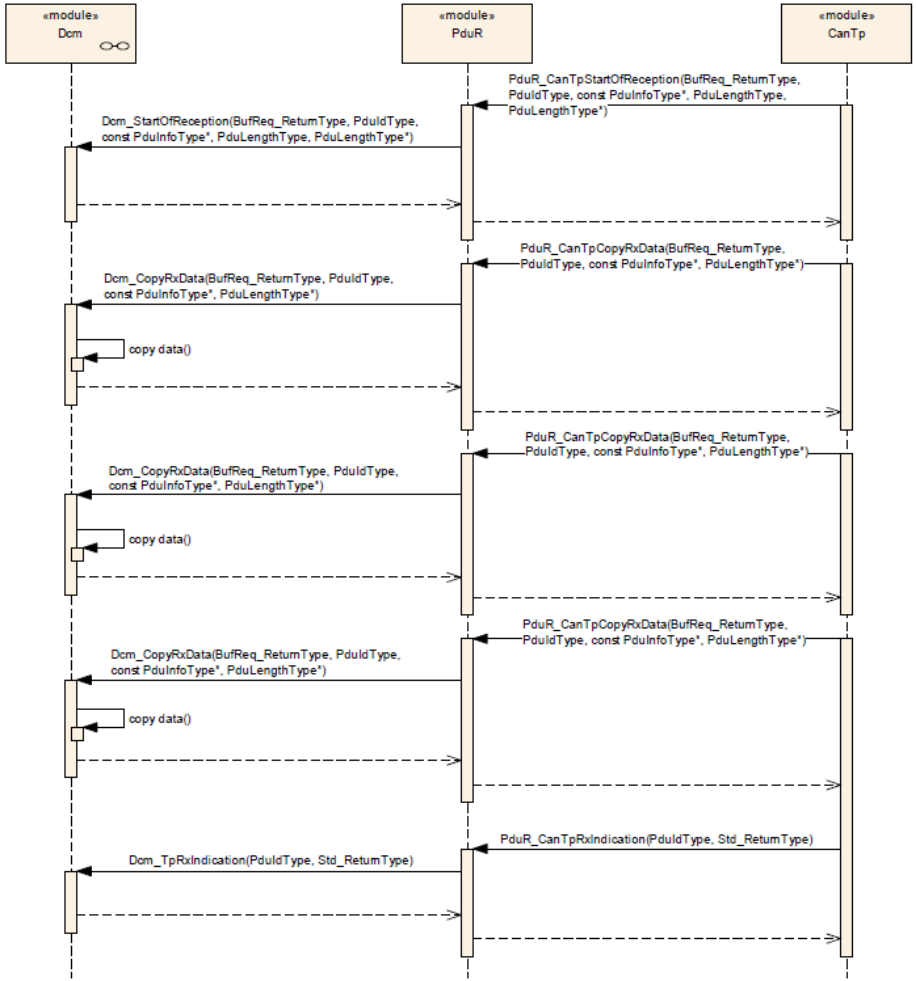


Figure 9.4: CanTp to Dcm TP PDU reception

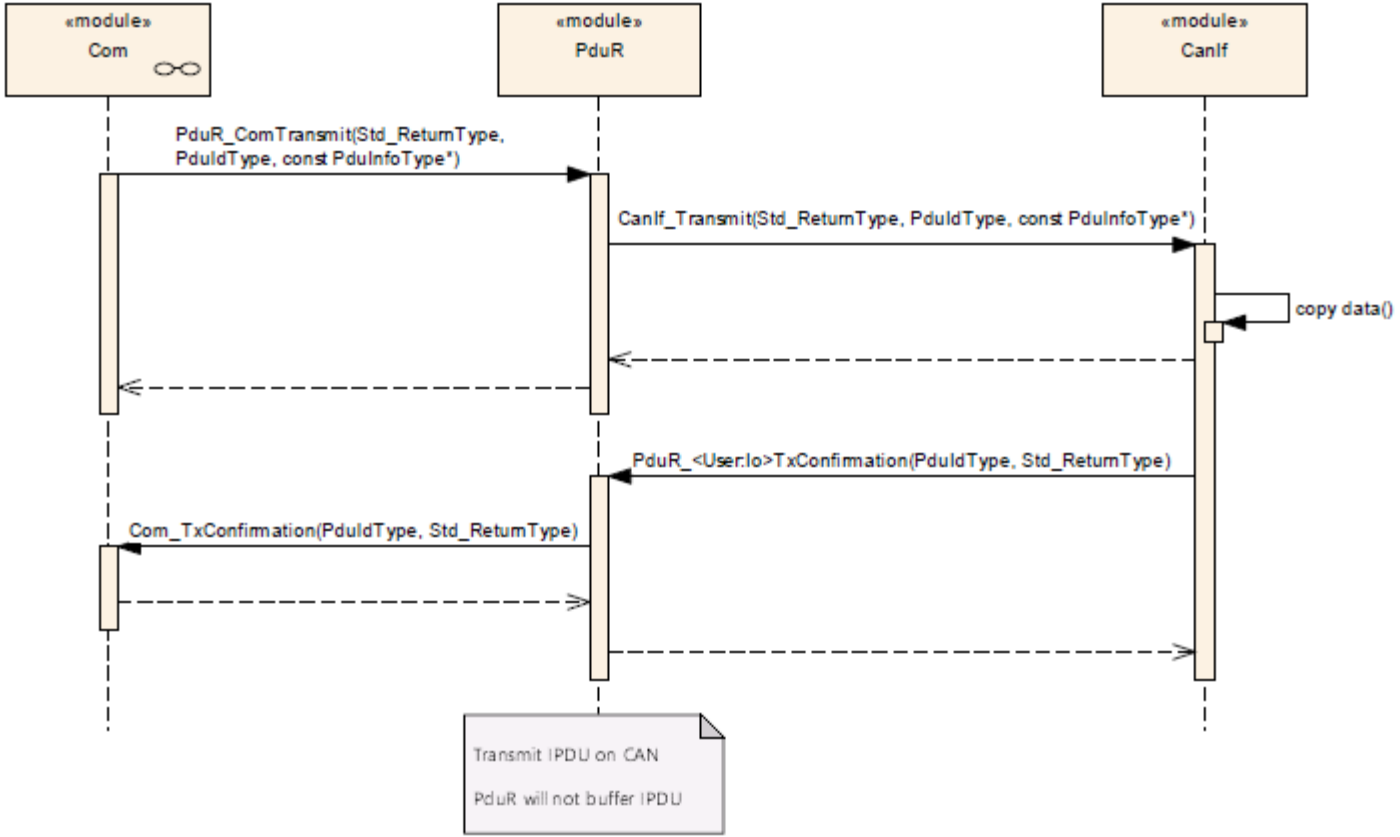


Figure 9.5: Com to CanIf I-PDU transmission

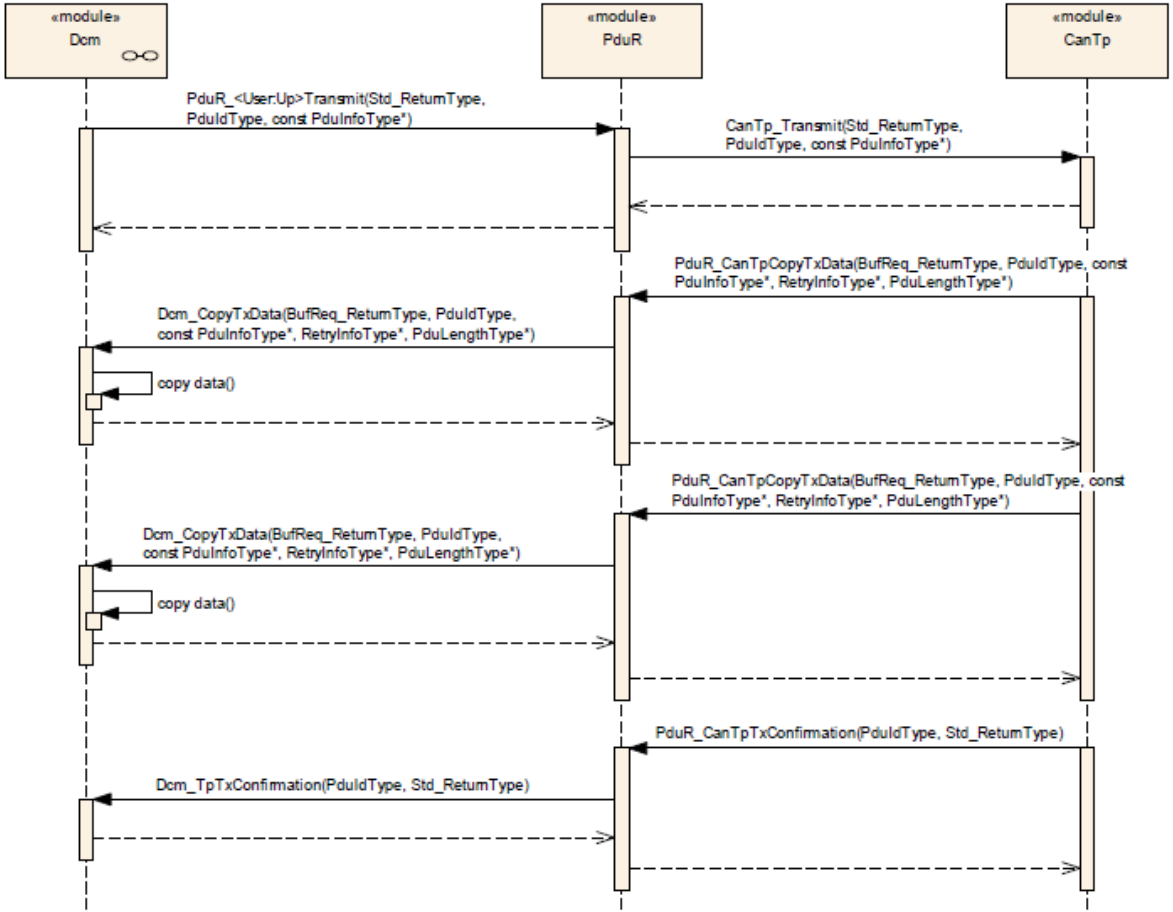


Figure 9.9: Dcm to CanTp PDU transmission

Configuration Specification

PduRBswModules	Each container describes a specific BSW module (upper/CDD/lower/IpduM) that the PDU Router shall interface to.
PduRGeneral	This container is a subcontainer of PduR and specifies the general configuration parameters of the PDU Router.
PduRRoutingPaths	Represents one table of routing paths.This routing table allows multiple configurations that can be used to create several routing tables in the same configuration.

PduR BswModules

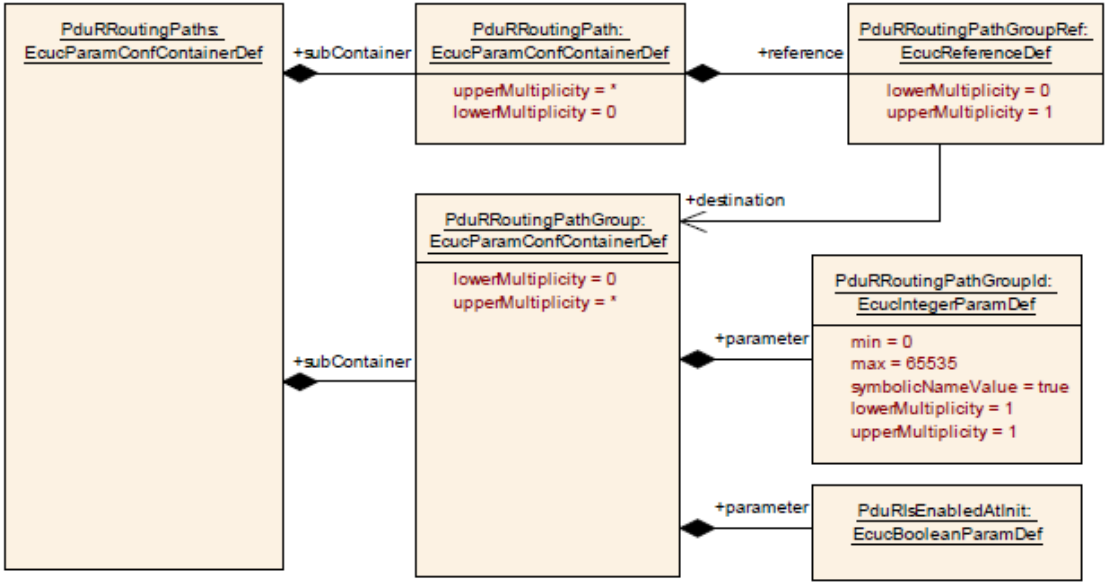
PduRCancelReceive	Specifies if the Transport protocol module supports the CancelReceive API or not. Value true the API is supported.
PduRCancelTransmit	Specifies if the BSW module supports the CancelTransmit API or not. Value true the API is supported.
PduRCommunicationInterface	Specifies if the BSW module supports the Communication Interface APIs or not. Value true the APIs are supported.
PduRCopyRxData	Specifies if the Transport protocol module supports the CopyRxData API or not. Value true the API is supported.
PduRCopyTxData	Specifies if the Transport protocol module supports the CopyTxData API or not. Value true the API is supported.
PduRLowerModule	The PduRLowerModule will decide who will call the APIs and who will implement the APIs.
PduRRetransmission	If set to true this means that the destination transport protocol module will use the retransmission feature.
PduRRxIndication	Specifies if BSW module supports the RxIndication API or not. Value true the API is supported.

PduRStartOfReception	Specifies if the Transport protocol module supports the StartOfReception API or not. true -> API is supported.
PduRTpRxIndication	Specifies if the Transport protocol module supports the TpRxIndication API or not. true -> API is supported.
PduRTpTransmit	Specifies if BSW module supports the TP Transmit API or not. true -> API is supported.
PduRTpTxConfirmation	Specifies if the Transport protocol module supports the TpTxConfirmation API or not. true -> API is supported.
PduRTransmit	Specifies if BSW module supports the (IF) Transmit API or not. true -> API is supported.
PduRTransportProtocol	The PDU Router module shall use the API parameters specified for transport protocol interface.
PduRTriggertransmit	Specifies if the BSW module supports the TriggerTransmit API or not.
PduRTxConfirmation	Specifies if the BSW module supports the TxConfirmation API or not. true -> API is supported.
PduRUpperModule	The PduRUpperModule will decide who will call the APIs and who will implement the APIs.
PduRUseTag	This parameter, if set to true, enables the usage of the tag (<up>) in the following API calls: PduR_<Up>CancelReceive, PduR_<Up>CancelTransmit
PduRBswModuleRef	This is a reference to one BSW module’s configuration

PduRGeneral

PduRDevErrorDetect	Switches the development error detection and notification on or off.
PduRMetaDataSupport	Enable support for MetaData handling. The MetaData is defined by the referenced MetaDataType of the global PDU definitions.
PduRVersionInfoApi	If true the PduR_GetVersionInfo API is available.
PduRZeroCostOperation	If set the PduR configuration generator will report an error if zero-cost-operation cannot be fulfilled.

PduRRoutingPathGroup



PduRIsEnabledAtInit	If set to true this routing path group will be enabled after initializing the PDU Router module (i.e. enabled in the PduR_Init function).
PduRRoutingPathGroupId	Identification of the routing group. The identification will be used by the disable/enable API in the PDU Router module API.

PduRRoutingPaths

PduRConfigurationId	Identification of the configuration of the PduR configuration. This identification can be read using the PduR API.
PduRMaxRoutingPathCnt	Maximum number of RoutingPaths in all RoutingTables. This parameter is needed only in case of post-build loadable implementation using static memory allocation.
PduRMaxRoutingPathGroupCnt	Maximum number of RoutingPathGroups. This parameter is needed only in case of post-build loadable implementation using static memory allocation.
PduRDestPdu	Specifies one destination for the PDU to be routed.
PduRRoutingPath	Specifies the routing path of a PDU.
PduRRoutingPathGroup	This container groups routing paths. By this grouping, itis possible to switch all routings related to one network, or to one kind of PDUs. PduRRoutingPaths link one source with one destination.
PduRSrcPdu	Specifies the source of the PDU to be routed.
PduRTxBuffer	Specifies a buffer used for gatewaying via communication interfaces or transport protocols.

PduRRoutingPath

PduRQueueDepth	This parameter defines the queue depth for this routing path.
PduRTpThreshold	This parameter is only relevant for TP routings.
PduRDestTxBufferRef	Reference to a buffer in the PduR. This buffer is required for communication interface gatewaying, and for transport protocol gatewaying.
PduRRoutingPathGroupRef	Reference to routing paths.
PduRSrcPduRRef	Reference to PduRSrcPdu

PduRDestPdu

PduRDestPduDataProvision	PDUR_DIRECT: The PDU Router module shall call the transmit function in the destination module and not buffer the I-PDU PDUR_TRIGGERTRANSMIT: The PDU Router module shall call the transmit function in the destination module. The dest module will request I-PDU using the triggerTransmit function.
PduRDestPduHandleId	PDU identifier assigned by PDU Router. Used by communication interface and transport protocol modules for confirmation (PduR_<Lo>TxConfirmation) and for TriggerTransmit (PduR_<Lo>TriggerTransmit).
PduRTransmissionConfirmation	This parameter is only for communication interfaces. Transport protocol modules will always call the TxConfirmation function.
PduRDestPduRef	Destination PDU reference; reference to unique PDU identifier which shall be used by the PDU Router instead of the source PDU ID when calling the related function of the destination module.

PduRSrcPdu

PduRSourcePduBlockSize	Minimum amount of buffer space required by receiving transport protocol layer to continue reception.
PduRSourcePduHandleId	PDU identifier assigned by PDU Router.
PduRSrcPduUpTxConf	When enabled, the TxConfirmation will be forwarded to the upper layer. Prerequisites: Lower layer and upper layer support TxConfirmation.
PduRSrcPduRef	Source PDU reference; reference to unique PDU identifier which shall be used for the requested PDU Router operation.

PduRDefaultValueElement

PduRDefaultValueElement	The default value consists of a number of elements. Each element is one byte long and the number of elements is specified by SduLength. The position of this parameter in the container is specified by the PduRElementBytePosition parameter.
PduRDefaultValueElementBytePosition	specifies the byte position of the element within the default value

PduRTxBuffer

PduRTxBuffer	Specifies a buffer used for gatewaying via communication interfaces orb transport protocols.
PduRPduMaxLength	Length of the Tx buffer in bytes. This parameter limits the size of buffered routed PDUs.